Evidence across a boundary

Simon Frost

Table of contents

1 Overview 1

2 Setup 2

3 Evidence upstream: the year's weather 3

4 Evidence downstream: the species was not found 4

5 Normalising each component is not sound 5

6 Every cut gives the same answer 6

7 Summary 6

1 Overview

An observation made in one part of a composed problem changes what is believed, and what is best, in the others. A weather record from the site changes the value of a grazing decision taken by managers who never see it. A survey that finds the species absent says something about the soil moisture upstream. This vignette composes the zoo's reference habitat network with a management decision, as the second and fourth vignettes do, and conditions it on evidence held by one component at a time, following ADR 0021.

The evidence is never normalised inside a component. A component with evidence has a reward kernel whose probabilities sum to less than one: they are the probability of each outcome *and* the evidence. That shortfall is what carries the evidence across a boundary. Conditioning each component on its own evidence and renormalising discards it, and the composite then ignores the evidence altogether.

2 Setup

The chain is the second vignette's, with a manager who does not measure soil moisture: one action for every site.

```
using CategoricalInfluenceDiagrams
using EcologicalBayesianNetworks

habitat = load_model("reference_habitat_bn")
grazing = load_model("reference_grazing_id")
zoo_kernels = Dict{Symbol,FiniteKernel}(x ⇒ kernel(habitat, x)
                                        for x in variable_names(syntax(habitat)))
moisture = [:low, :medium, :high]
nothing # hide
```

Four components:

- site: climate and irrigation, generating the soil moisture, which leaves it hidden;
- management: the decision `GrazingManagement` (exclude, reduce or maintain), with no information, the grazing pressure it achieves and its cost;
- biotic: vegetation, habitat quality and occupancy, from soil moisture and grazing pressure;
- valuation: the species' presence, valued at 1000.

```
abiotic = bayesnet(:Climate ⇒ [:dry, :normal, :wet], :Irrigation ⇒ [:low, :high],
                   :SoilMoisture ⇒ moisture;
                   mechanisms = [:SoilMoisture ⇒ (:Climate, :Irrigation)])
site = Open(InfluenceDiagram(abiotic); outputs = [:SoilMoisture])
management = Open(influence_diagram(:SoilMoisture ⇒ moisture,
                                    :GrazingManagement ⇒ [:exclude, :reduce,
                                    :maintain],
                                    :GrazingPressure ⇒ [:low, :high];
                                    mechanisms = [:GrazingPressure ⇒
:GrazingManagement],
                                    decisions = [:GrazingManagement ⇒ Symbol[]],
                                    utilities = [:ManagementCost ⇒
:GrazingManagement],
                                    closed = false);
                  inputs = [:SoilMoisture], outputs = [:SoilMoisture,
:GrazingPressure])
biotic = Open(InfluenceDiagram(bayesnet(:SoilMoisture ⇒ moisture,
                                        :GrazingPressure ⇒ [:low, :high],
                                        :Vegetation ⇒ [:sparse, :moderate, :dense],
                                        :HabitatQuality ⇒ [:poor, :good],
                                        :Occupancy ⇒ [:absent, :present];
                                        mechanisms = [:Vegetation ⇒ (:SoilMoisture,
                                                                     :
                                                                     GrazingPressure),
                                                      :HabitatQuality ⇒
                                                      :Vegetation,
```

```
                                                    :Occupancy ⇒
                                                    :HabitatQuality],
                                          closed = false));
             inputs = [:SoilMoisture, :GrazingPressure], outputs = [:Occupancy])
valuation = Open(influence_diagram(:Occupancy ⇒ [:absent, :present];
                                   utilities = [:ConservationBenefit ⇒ :Occupancy],
                                   closed = false);
                 inputs = [:Occupancy])
chain = [site, management, biotic, valuation]
whole = foldl(compose, chain)
kernels = merge(zoo_kernels, Dict(:GrazingPressure ⇒ kernel(grazing,
:GrazingPressure)))
presence = FiniteAxis(:Occupancy, [:absent, :present])
utilities = Dict{Symbol,AbstractUtility}(:ManagementCost ⇒
bound_utilities(grazing)[:ManagementCost],
                                         :ConservationBenefit ⇒
                                         TabularUtility([presence],

[0.0, 1000.0]))
act(a) = Strategy(:GrazingManagement ⇒ ConstantPolicy(a))
actions = (:exclude, :reduce, :maintain)
nothing # hide
```

Without evidence, reducing grazing is narrowly the best of the three actions:

```
[a ⇒ round(contextual_value(whole, kernels, utilities, act(a)); digits = 2)
 for a in actions]
```

```
3-element Vector{Pair{Symbol, Float64}}:
  :exclude => 468.12
   :reduce => 468.32
 :maintain => 451.45
```

3 Evidence upstream: the year's weather

The site component holds the climate. Suppose the year is known to be wet. With the `evidence` keyword, `conditional_value` is the expected utility given that evidence, and it changes the best action: in a wet year excluding grazing pays.

```
wet = :Climate ⇒ :wet
[a ⇒ round(conditional_value(whole, kernels, utilities, act(a); evidence = wet);
           digits = 2)
 for a in actions]
```

```
3-element Vector{Pair{Symbol, Float64}}:
  :exclude => 517.19
   :reduce => 514.96
 :maintain => 494.95
```

The managers do not observe the weather, and do not need to: the evidence is held by the site component, and it reaches the decision through the boundary. Pushed forward through the site, it becomes a *state* on the soil moisture: the unnormalised law of each soil moisture state together with the evidence. Its mass is the probability of a wet year, and divided by that mass it is the posterior law of the soil moisture.

```
s = boundary_state(site, kernels, utilities, Strategy(); evidence = wet)
ν = vec(kernel(s).table)
(prior = round.(boundary_law(site, kernels, utilities, Strategy()); digits = 3),
 mass = round(sum(ν); digits = 3), posterior = round.(ν ./ sum(ν); digits = 3))
```

```
(prior = [0.288, 0.403, 0.309], mass = 0.2, posterior = [0.08, 0.34, 0.58])
```

Pulled back through the biotic and valuation components, the rest of the chain is an *effect* on the management component's outputs. Here it carries no evidence, so its probability row is one. The management component optimised between the state and the effect makes the same choice as the closed composite given the evidence, with the same value:

```
downstream = continuation_effect(compose(biotic, valuation), kernels, utilities,
Strategy())
in_context = contextual_optimize(management, kernels, utilities; prior = s,
                                 continuation = downstream)
closed = optimize(whole, kernels, utilities; evidence = wet)
(in_context = (round(in_context.expected_utility; digits = 2),
               only(policy_table(in_context.strategy[:GrazingManagement]))),
 closed = (round(closed.expected_utility; digits = 2),
           only(policy_table(closed.strategy[:GrazingManagement]))))
```

```
(in_context = (517.19, :exclude), closed = (517.19, :exclude))
```

4 Evidence downstream: the species was not found

Now suppose a survey at the end of the season finds the species absent. The evidence is held by the biotic component, downstream of the decision. Pulled back to the boundary between the site and the management component, the rest of the chain is an effect whose probability row is the *likelihood* of the absence given each soil moisture state, under the action taken. Multiplied by the prior law of the soil moisture and normalised, it is the posterior at that boundary:

```
absent = :Occupancy ⇒ :absent
prior = boundary_law(site, kernels, utilities, Strategy())
rest(a) = continuation_effect(foldl(compose, chain[2:4]), kernels, utilities,
act(a);
                              evidence = absent)
[a ⇒ (likelihood = round.(vec(kernel(rest(a)).table); digits = 3),
      posterior = round.(prior .* vec(kernel(rest(a)).table) ./
                         sum(prior .* vec(kernel(rest(a)).table)); digits = 3))
 for a in actions]
```

```
3-element Vector{Pair{Symbol, @NamedTuple{likelihood::Vector{Float64},
posterior::Vector{Float64}}}}:
  :exclude => (likelihood = [0.608, 0.482, 0.397], posterior = [0.356, 0.395, 0.25])
   :reduce => (likelihood = [0.627, 0.506, 0.427], posterior = [0.35, 0.395, 0.255])
 :maintain => (likelihood = [0.653, 0.538, 0.465], posterior = [0.343, 0.395,
 0.262])
```

Low soil moisture makes the absence likelier, so the posterior moves towards dry sites: the survey downstream is evidence about the site upstream, and the likelihood crossed the boundary backwards to say so. The posterior also depends on the action. Under heavier grazing an absence says less about the soil, because grazing alone could explain it.

Evidence downstream of a decision is a retrospective question, not information for the decision. Optimising the conditional value given the absence asks which action is best *given that the species turned out absent*: none of them earns the benefit then, so the cheapest wins.

```
given_absence = optimize(whole, kernels, utilities; evidence = absent,
                         backend = ExhaustivePolicySearch())
round(given_absence.expected_utility; digits = 2),
only(policy_table(given_absence.strategy[:GrazingManagement]))
```

```
(0.0, :maintain)
```

5 Normalising each component is not sound

The evidence mass must cross the boundary. Normalise the downstream effect on its own evidence, dividing each entry by its probability, and the likelihood is gone: every soil moisture state now predicts the absence with probability one, and the posterior at the boundary is the prior again.

```
effect = rest(:reduce)
m = vec(kernel(effect).table)
normalised = RewardKernel(FiniteKernel(dom(effect), codom(effect),
                                       reshape(m ./ m, size(kernel(effect).table))),
                          reshape(vec(reward(effect)) ./ m, size(reward(effect))))
posterior(ef) = prior .* vec(kernel(ef).table) ./ sum(prior .*
vec(kernel(ef).table))
(prior = round.(prior; digits = 3), carried = round.(posterior(effect); digits = 3),
 normalised = round.(posterior(normalised); digits = 3))
```

```
(prior = [0.288, 0.403, 0.309], carried = [0.35, 0.395, 0.255], normalised = [0.288,
0.403, 0.309])
```

The Lean module `Evidence` proves the smallest case of this: a uniform binary state worth 1 when true, and evidence nine times likelier when it is true, has conditional value `9/10`; the components normalised one by one give `1/2` (`local_normalisation_obstruction`).

6 Every cut gives the same answer

With evidence at both ends, the wet year upstream and a vegetation survey downstream that records dense vegetation, the evidence mass and the conditional value can be computed in the context of any component: the state pushed forward to its inputs through the components before it, and the effect pulled back to its outputs through the components after it. For the management, biotic and valuation components they agree with each other and with the closed composite:

```
both = [wet, :Vegetation ⇒ :dense]
σ = act(:reduce)
part_evidence(o) = [x ⇒ v for (x, v) in both if has_variable(apex(o), x)]
r = [interpret(o, kernels, utilities, σ; evidence = part_evidence(o)) for o in
chain]
state_before(j) = foldl((st, ri) -> boundary_state(ri; prior = st), r[2:(j - 1)];
                        init = boundary_state(r[1]))
effect_after(j) = j == length(r) ? nothing :
                  foldr((ri, ef) -> continuation_effect(ri; continuation = ef),
                        r[(j + 1):(end - 1)]; init = continuation_effect(r[end]))
at_cut = map(2:4) do j
    st, ef = state_before(j), effect_after(j)
    (mass = round(evidence_mass(r[j]; prior = st, continuation = ef); digits = 6),
     value = round(conditional_value(r[j]; prior = st, continuation = ef); digits =
6))
end
whole_value = round(conditional_value(whole, kernels, utilities, σ; evidence =
both);
                    digits = 6)
(at_cut = at_cut, closed = whole_value)
```

```
(at_cut = [(mass = 0.07676, value = 652.5), (mass = 0.07676, value = 652.5), (mass =
0.07676, value = 652.5)], closed = 652.5)
```

The two pairings, the evidence mass `Σ ν m` and the evidence-weighted value `Σ (ρ m + ν c)` of a state `(ν, ρ)` against an effect `(m, c)`, are the same on either side of every component: the Lean theorems `mass_push` and `value_push`, with no normalisation hypothesis.

7 Summary

Evidence held by one component of a composed decision problem is carried across each boundary as mass. Forward, a state holds the probability of the evidence seen so far. Backward, an effect holds the likelihood of the evidence still to come. A wet year, recorded at the site, changes the managers' best action without their seeing it. An absence, recorded by the survey, moves belief about the site's soil moisture. Dividing the evidence out of a component before composing throws the evidence away. `boundary_state`, `continuation_effect`, `evidence_mass` and `conditional_value` compute these contexts and values component by component. `contextual_optimize` and `optimize` take the evidence too.